

RELATÓRIO : PILHA MIN-MAX

Aluno: Diogo Soares Germano

Análise estrutural da classe PilhaMinMax

1. ESTRUTURA DA SOLUÇÃO

A classe otimiza a busca pelos valores mínimo e máximo em uma pilha sem realizar varreduras lineares. A estratégia baseia-se na alocação de espaço adicional em memória para eliminar o tempo de busca em tempo de execução, utilizando três pilhas internas da API padrão do Java (`java.util.Stack`):

- **Pilha Principal (pilha):** Armazena todos os elementos inseridos (fluxo LIFO padrão).
- **Pilha Mínima (pilhaMin):** Armazena o histórico de valores mínimos. O topo reflete o menor valor atual.
- **Pilha Máxima (pilhaMax):** Armazena o histórico de valores máximos. O topo reflete o maior valor atual.

2. COMPLEXIDADE DE TEMPO

Método	Complexidade	Classificação
min()	$O(1)$	Tempo Constante
max()	$O(1)$	Tempo Constante

3. JUSTIFICATIVA

O desempenho de tempo constante $O(1)$ é garantido pelos seguintes fatores:

- 1. Acesso Direto ao Topo:** Os métodos `min()` e `max()` realizam apenas uma operação de inspeção (`peek`) nas pilhas auxiliares correspondentes.
- 2. Indexação em Vetor:** Como a classe `Stack` do Java utiliza internamente um array dinâmico, a operação `peek` consiste na leitura direta do último índice ativo da memória.
- 3. Antecipação do Custo Computacional:** O trabalho de comparação e atualização dos valores extremos é executado durante as operações de modificação (`push` e `pop`).

Assim, o custo é absorvido na inserção/remoção para manter as consultas instantâneas.

4. CÓDIGO (JAVA)

```
package org.univille;
import java.util.Stack;

public class PilhaMinMax {

    private Stack<Integer> pilha;
    private Stack<Integer> pilhaMin;
    private Stack<Integer> pilhaMax;

    public PilhaMinMax() {

        pilha = new Stack<>();
        pilhaMin = new Stack<>();
        pilhaMax = new Stack<>();
    }

    public void push(int valor) {

        pilha.push(valor);

        if (pilhaMin.isEmpty() || valor <= pilhaMin.peek()) {
            pilhaMin.push(valor);
        }

        if (pilhaMax.isEmpty() || valor >= pilhaMax.peek()) {
            pilhaMax.push(valor);
        }
    }

    public int pop() {

        if (pilha.isEmpty()) {
            throw new RuntimeException("Pilha vazia");
        }

        int removido = pilha.pop();

        if (removido == pilhaMin.peek()) {
            pilhaMin.pop();
        }

        if (removido == pilhaMax.peek()) {
            pilhaMax.pop();
        }

        return removido;
    }

    public int min() {

        if (pilhaMin.isEmpty()) {
            throw new RuntimeException("Pilha vazia");
        }

        return pilhaMin.peek();
    }

    public int max() {

        if (pilhaMax.isEmpty()) {
            throw new RuntimeException("Pilha vazia");
        }

        return pilhaMax.peek();
    }

    public void imprimirEstado() {

        System.out.println("Pilha: " + pilha);

        if (!pilha.isEmpty()) {
            System.out.println("Min: " + min());
            System.out.println("Max: " + max());
        }

        System.out.println("=====");
    }
}
```

```
package org.univille;
public class Main {

    public static void main(String[] args) {

        PilhaMinMax pilha = new PilhaMinMax();

        System.out.println("Inserindo elementos...");

        pilha.push(15);

        pilha.imprimirEstado();

        pilha.push(2);
        pilha.imprimirEstado();

        pilha.push(20);
        pilha.imprimirEstado();

        pilha.push(8);
        pilha.imprimirEstado();

        pilha.push(30);
        pilha.imprimirEstado();

        System.out.println("Removendo elementos...");

        pilha.pop();
        pilha.imprimirEstado();

        pilha.pop();
        pilha.imprimirEstado();

        pilha.pop();
        pilha.imprimirEstado();

    }
}
```